MOVIENIGHT PROJECT

Presented by Sergey Aitov, Alexander Rozanov, Dmitriy Prokopyev, Egor Dushin

PROBLEM

Memory safety vulnerabilities (e.g. buffer over-read) are a common and painful problem for web applications.

KEY RISK: An attacker can extract secrets (e.g. Heartbleed CVE) by probing the application memory through a vulnerability.

KEY CONSEQUENCES:

- Violation of confidentiality as a result of data exfiltration;
- Privelege escalation via credential-requiring services;
- Lateral movement in the system;
- Violation of integrity via stolen credentials.

PROJECT CONTEXT

GOAL: Implement a PoC in which the database encryption key is isolated from the main web application.

PROVIDE EVIDENCE THAT:

- Components responsible for business logic are intact;
- Credential exfiltration through compromised process memory is impossible;
- All credentials are isolated in a sufficiently secure contour;
- Side effects are tolerable or desirable.

APPLICATION

MovieNight

Movie Night is a web application for organizing movie screenings in a dormitory: users can register, create movie suggestions for rooms and time slots, view schedules, vote for conflicting screenings, and leave snack reactions for a selected movie.

STACK: FastAPI, SQLAlchemy, SQLite, PyJWT, Argon2id, Streamlit.

INITIAL APPROACH:

BACKEND → SQL ALCHEMY → SQLite

MODIFICATIONS

NEW APPROACH

MODIFICATION: Credential handling and business logic are now decoupled and operate independently.

EFFECT: The backend process does not store credentials in memory at any moment in time, direct exfiltration is impossible.

BACKEND → SQLITE-PROXY → SQL ALCHEMY → SQLite

SQLITE-PROXY

FUNCTIONS:

- Authenticating with Conjur Open Source
- Acquiring the database encryption key;
- Executing database operations.

INTERNAL API:

- ***/internal/auth/**** - user operations;
- ***/internal/proposals/**** - operations with movie proposals;
- ***/internal/votes/**** - operations with votes;
- ***/internal/reactions/**** - operations with reactions;
- ***/internal/reads/**** - read operations;

HASHICORP VAULT

FUNCTIONS:

- Generation of cryptographically strong encryption key;
- Secure storage of the encryption key;
- Rotation of the encryption key according to the policy.

PURPOSE: To be the singular source of truth for all secrets at any moment in time.

CONJUR OPEN SOURCE

FUNCTIONS:

- Strong machine identity verification;
- Policy-based provision of secrets to the chosen components;

POLICY: Only the proxy is allowed to read the encryption key. Nothing else can access the credentials.

ACCESS MODEL:

DEMO VIDEO

RESULTS

IMPROVED SECURITY

KEY RESULT: Database credentials leak can no longer be achieved by compromising the memory and environment of the main application.

REASONS:

- The encryption key does not enter the main app memory at any point in time;
- Communication between the main app, proxy, and support components is strictly limited to private docker network;
- Proxy encapsulated data-related business logic, standard abuse is impossible;
- Proxy has an extremely small attack surface, securing this one point is easier;
- Monitoring, logging, and hardening efforts can be concentrated on the proxy.

LIMITATIONS

KEY RESULT: This approach protects specifically against memory-read vulnerabilities, but has weaknesses elsewhere;

REASONS:

- If the docker host is compromised, isolation can be bypassed;
- Compromising the proxy itself is very difficult, but still possible;
- If main app's calls in the internal network can be falsified, then the security benefit dissipates;
- Private network isolation is an unavoidable and last line of defense;
- If the main app is exploited to make requests to the proxy, then the other defenses are meanigless.

OUR TEAM

Sergey Aitov

Alexander Rozanov

Dmitriy Prokopyev

Egor Dushin

THANK YOU